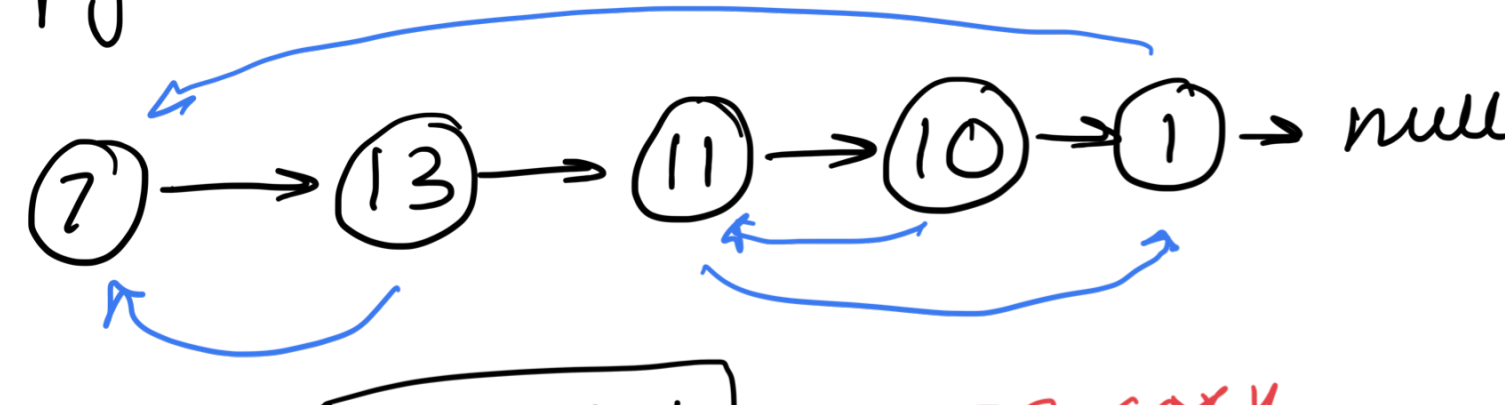


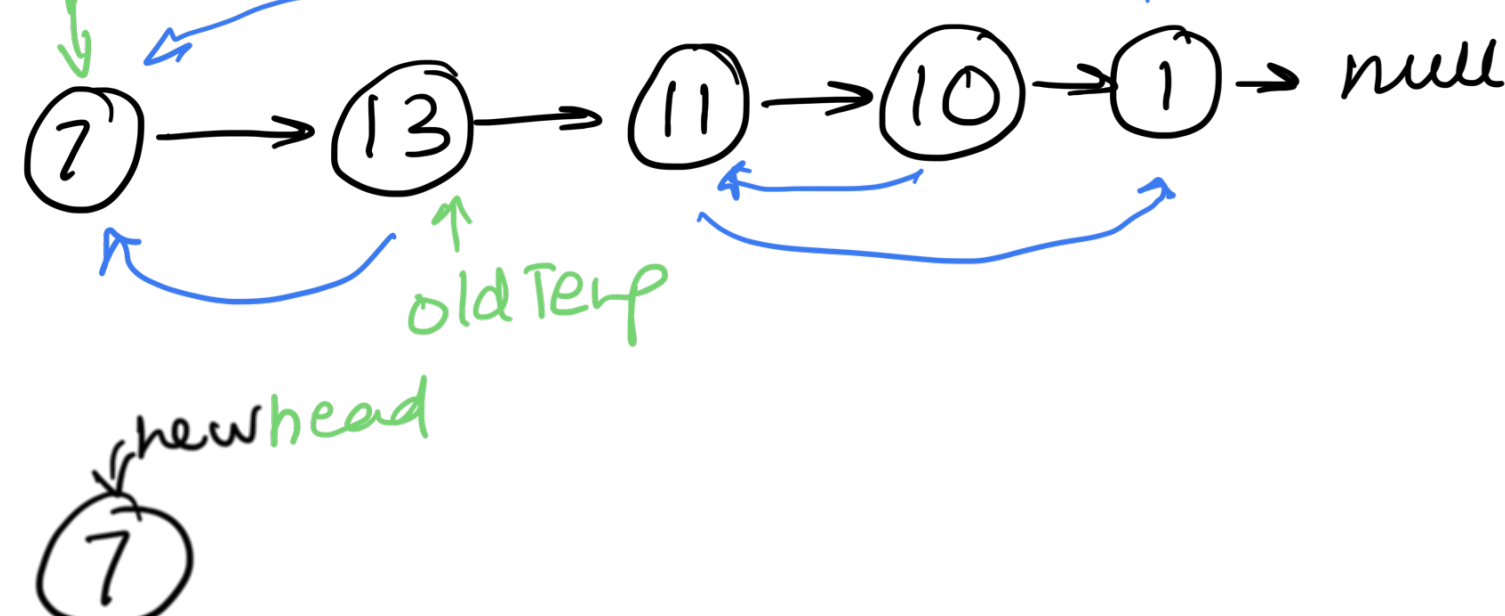
Copy list with random pointer



data/val
next
random

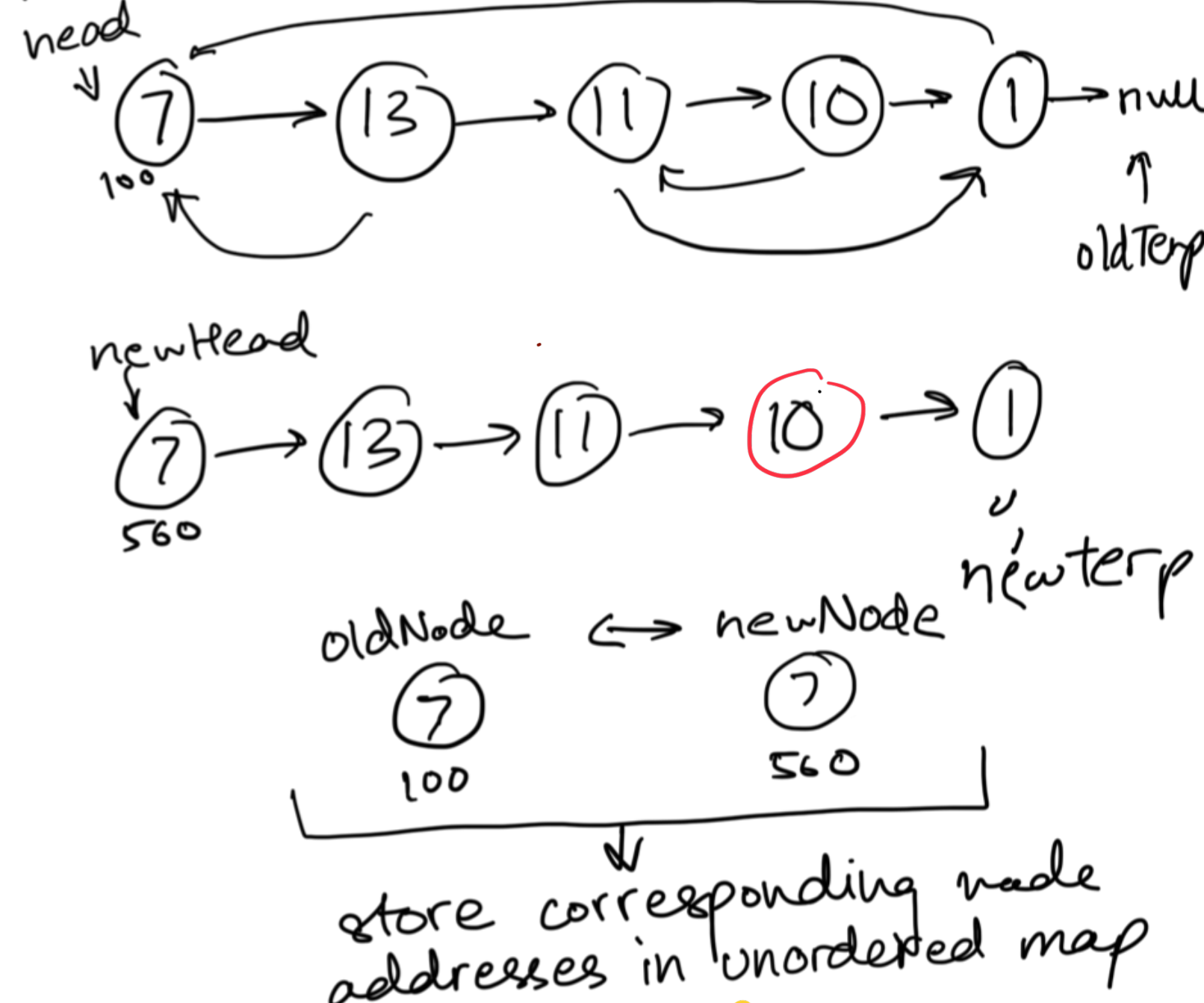
DEEP COPY
even the random connections must be replicated in copy

① have single copy ignore random



```
Node* newHead = new Node(head->val)
oldTemp = head->next
newTemp = newHead
while (oldTemp != NULL) {
    Node* copyNode = new Node(oldTemp->val)
    newTemp->next = copyNode
    oldTemp = oldTemp->next
    newTemp = newTemp->next
}
```

② recreate random connectors



oldNode ↔ newNode
store corresponding node addresses in unordered map

Why?

in the original list we know that 1 points to 7
so after we have made the copy we want to recreate the connections right? so we store the purana 7 ka address and new 7 ka address in an unordered map.

- the 1 points to 7
- check 7 in map and use a copy
- fetch address of new 7 and create connection

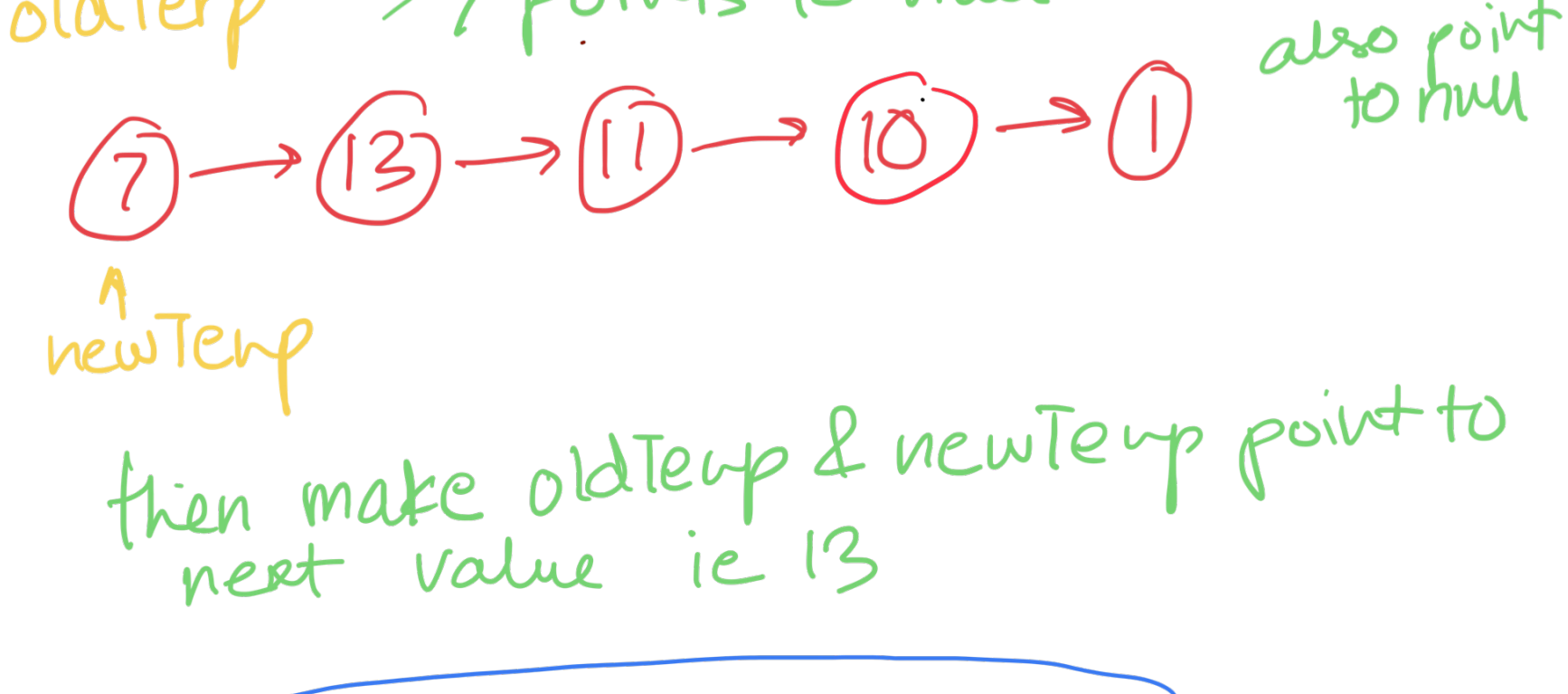
unordered_map <N, N> m;
oldNode ↑ newNode

old (key)	new (val)
7	7
13	13
11	11
10	10
1	1

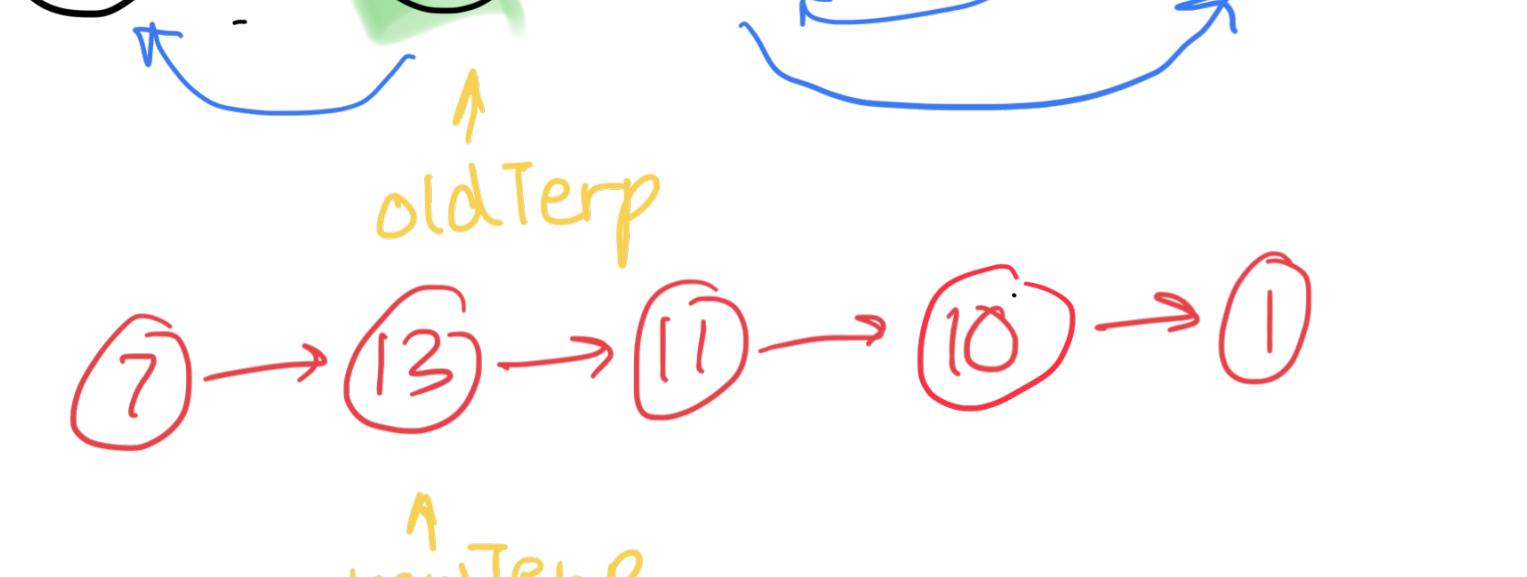
← storing pointer (addresses)

```
unordered_map <N, N> m,
Node* newHead = new Node(head->val)
oldTemp = head->next
newTemp = newHead
m[head] = newHead
while (oldTemp != NULL) {
    Node* copyNode = new Node(oldTemp->val)
    m[oldTemp] = copyNode
    newTemp->next = copyNode
    oldTemp = oldTemp->next
    newTemp = newTemp->next
}
```

oldTemp = head, newTemp = newHead



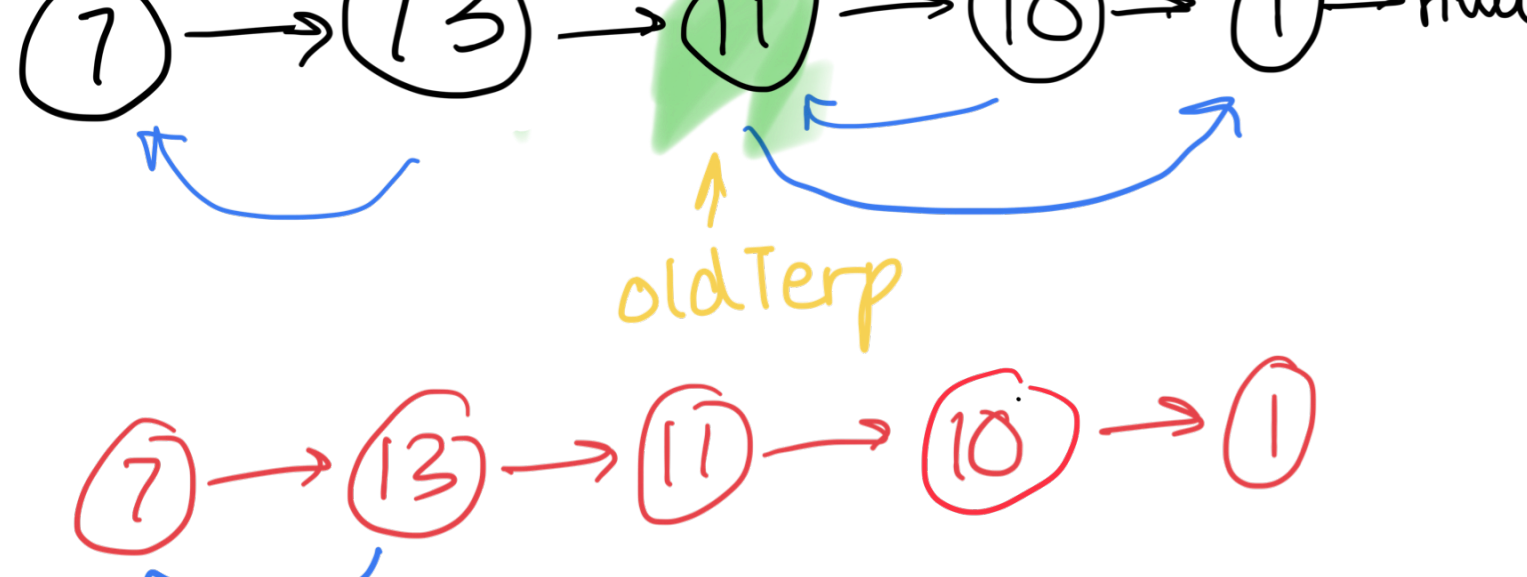
then make oldTemp & newTemp point to next value i.e 13



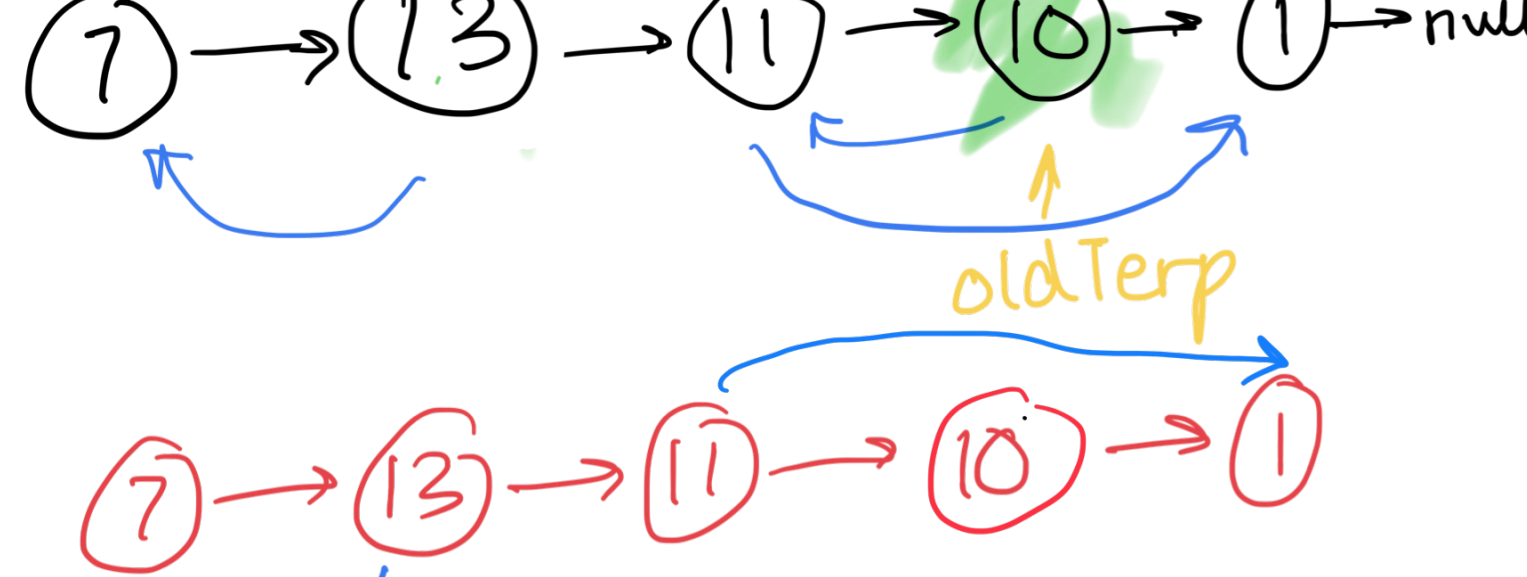
to make new 13 point to new 7 we must know address of new 7 but we don't have that, that's exactly why he needed an unordered map to store addresses

old (key)	new (val)
7	7
13	13
11	11
10	10
1	1

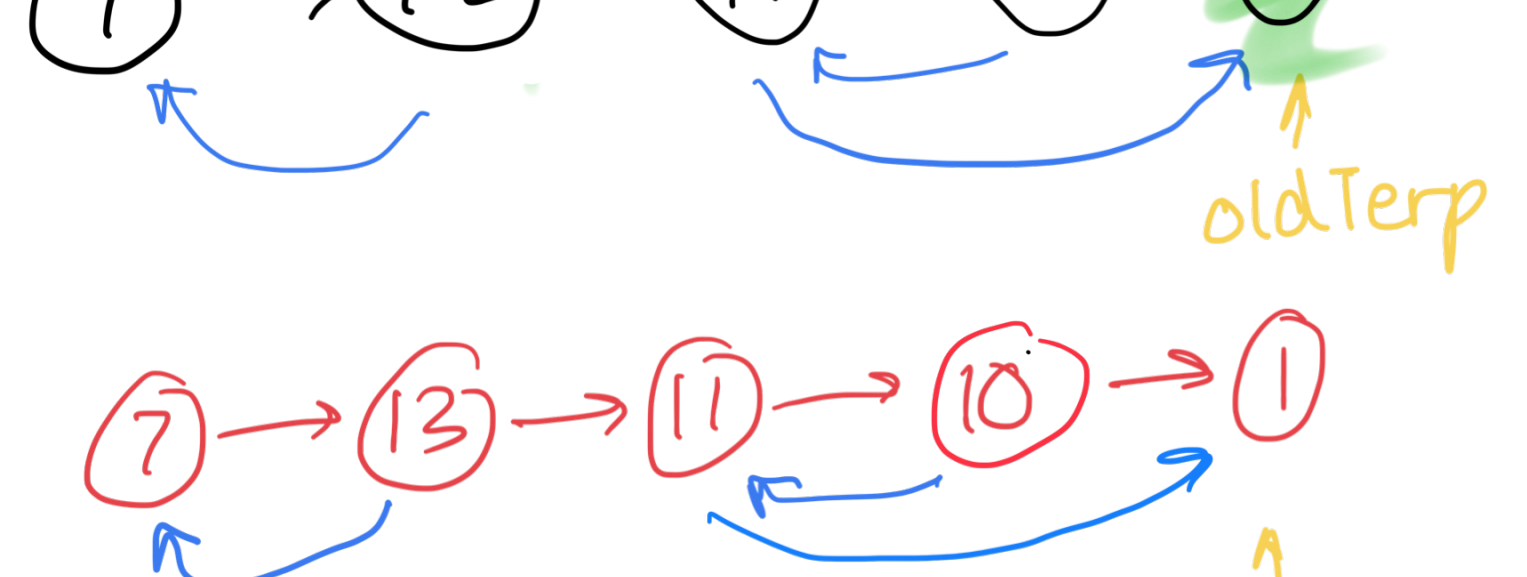
therefore we connect new 13 to new 7



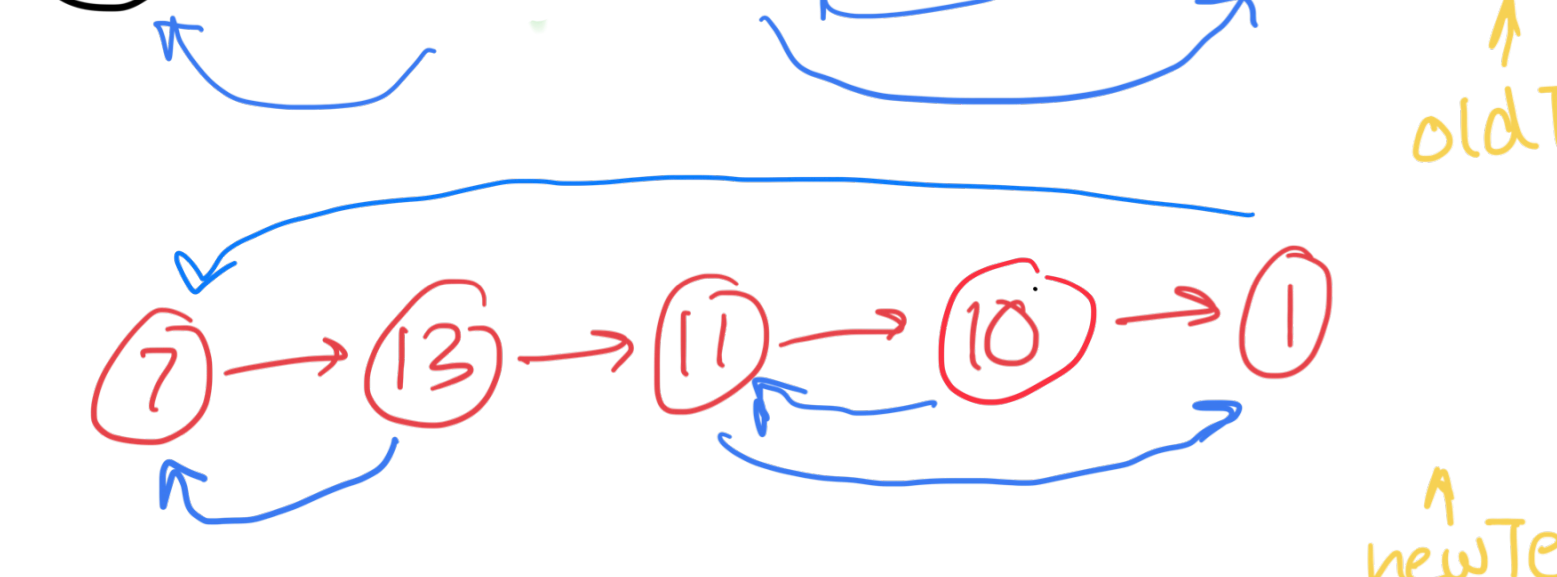
old 11 points to old 1
1 ka address we know thanks to the map so just make connection



old 10 points to old 11
we know new 11 addy becu of map
make connection



we know old 1 points to old 7
we know new 7 addy
make connection



```
oldTemp = head
newTemp = newHead
while (oldTemp != null) {
    newTemp->random = m[oldTemp->random]
    oldTemp = oldTemp->next
    newTemp = newTemp->next
}
```